

KJO Liquid Rocket Motor Control System

System Architecture and Operational Overview

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.1

Date: March 2026

This document provides a high-level structural and functional overview of the KJO control system for a bi-propellant liquid rocket motor. It describes the four electronic subsystems (RCU, EMU, GSEMU, LCMU), the propellant flow architecture, the inter-unit communication protocols, and the complete command set. Detailed subsystem reference documents are listed in [Section 6](#).

Contents

1	Introduction	2
2	Propellant System	2
2.1	Flow Architecture	2
2.2	Fill Sequence	3
2.3	Engine Firing Sequence	4
2.4	Dump / Safe Procedure	4
2.5	Pressurization	4
3	Electronic Control Architecture	4
3.1	RCU-EMU Radio Link	5
3.2	EMU-GSEMU-LCMU CAN Bus	5
4	Shared Libraries	5
5	Command Set	5
5.1	Packet Structures	5
5.2	Command Table	6
6	Document References	6
7	Version History	6

1 Introduction

The KJO system controls a bi-propellant liquid rocket motor using nitrous oxide (N_2O) as the oxidizer and a liquid fuel with regenerative cooling. Four autonomous microcontroller units share control responsibilities:

- **Remote Control Unit (RCU)** — handheld operator terminal. Sends commands over a 900 MHz LoRa radio link; displays system status on a 3.5" colour touchscreen.
- **Engine Management Unit (EMU)** — mounted in the rocket. Receives commands from the RCU, actuates the Oxidizer Feed and Fuel Feed servos, reads all sensors, and coordinates the Fill valve and load-cell readings with the GSEMU and LCMU over a shared CAN bus.
- **GSE Management Unit (GSEMU)** — mounted on the launch pad. Controls the Fill valve servo and the umbilical quick-release servo; coordinates the fill-to-target sequence with the LCMU and reports status to the EMU, both over the shared CAN bus. A separate, ground-referenced disconnect-sense line (not CAN) gives both EMU and GSEMU instant, protocol-independent detection of umbilical separation.
- **Load Cell Management Unit (LCMU)** — mounted on the test stand. Measures Ox load during oxidizer fill (supporting GSEMU's fill-to-target sequence) and thrust during test-stand burns, logging both to microSD; answers CAN queries from EMU and GSEMU.

RCU, EMU, and GSEMU are based on the Adafruit Feather M0 (ATSAMD21G18A, 48 MHz ARM Cortex-M0+, 3.3 V logic); LCMU is based on the Adafruit Feather RP2040 Adalogger (dual-core Cortex-M0+, 133 MHz). All four run firmware written in C++17 using the Arduino / PlatformIO framework.

2 Propellant System

2.1 Flow Architecture

Figure 1 shows the propellant system schematic.

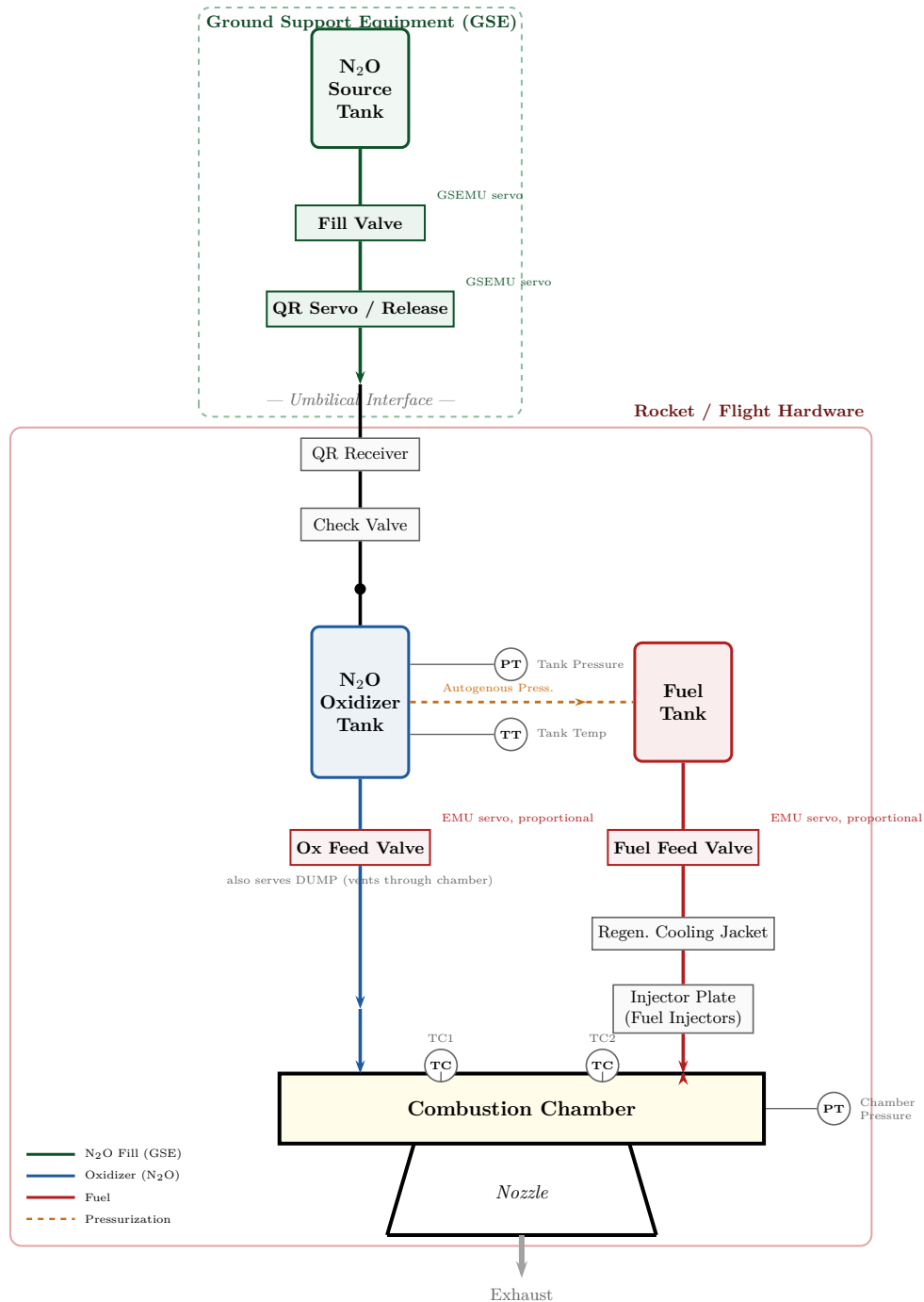


Figure 1: Propellant system schematic. Green lines: N₂O fill path (GSE). Blue lines: oxidizer feed path. Red lines: fuel feed path. Orange dashed: autogenous pressurization.

2.2 Fill Sequence

1. Operator commands **BEGIN_FILL** from the RCU.
2. EMU asserts the CMD bit (**AUX_IO_1**) high.
3. GSEMU detects the CMD rising edge and opens the Fill valve servo.
4. N₂O flows: GSE source tank → Fill valve → QR servo → umbilical → check valve →

on-board oxidizer tank.

5. GSEMU asserts STATUS (AUX_IO_2) high when fill is complete.
6. EMU de-asserts CMD; GSEMU closes the Fill valve.

2.3 Engine Firing Sequence

1. Operator commands **ENABLE_LCO_CONTROL** (enables launch authority).
2. Operator commands **START_ENGINE**, which calls **Execute_Command(START_ENGINE)**.
3. EMU simultaneously opens Ox Feed valve and Fuel Feed valve to programmed positions and activates the igniter relay.
4. Proportional control commands (**OXIDIZER_FLOW_PROP**, **FUEL_FLOW_PROP**) may be sent during burn to adjust mixture ratio.
5. **STOP_ENGINE_FLOWS** closes both feed valves to terminate thrust.

2.4 Dump / Safe Procedure

The **DUMP** command opens the Oxidizer Feed valve (EMU-controlled), venting the on-board N_2O tank pressurant through the combustion chamber. There is no dedicated Dump valve. The RCU Dump button has latching behaviour — a second press sends **STOP_DUMP** and closes the Oxidizer Feed valve.

2.5 Pressurization

The on-board N_2O tank is self-pressurizing (autogenous). The vapour-space pressure is communicated to the fuel tank via a dedicated pressurization line, eliminating the need for a separate pressurant.

3 Electronic Control Architecture

Figure 2 shows the four-unit electronic control architecture.

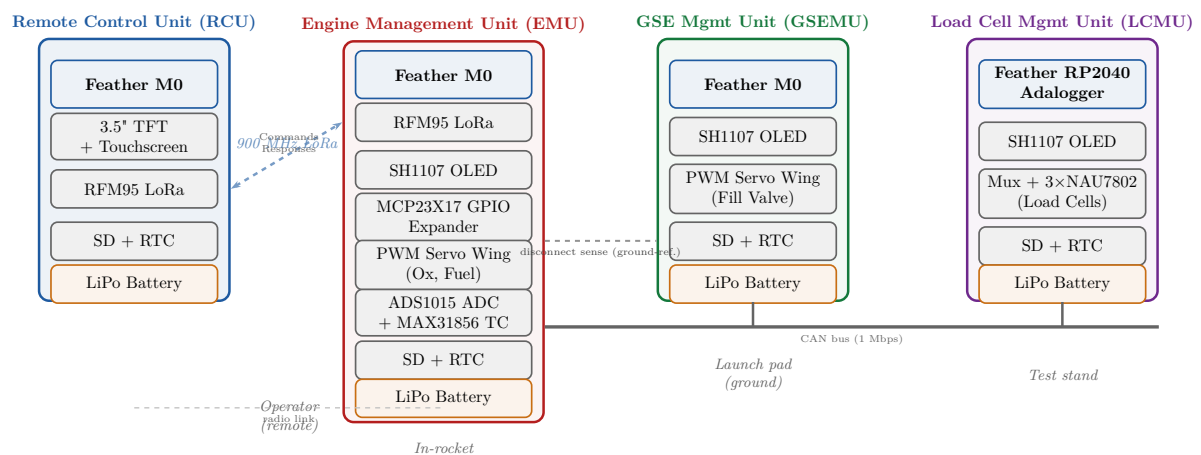


Figure 2: Electronic control architecture. The RCU communicates with the EMU over a 900 MHz LoRa radio link. EMU, GSEMU, and LCMU share a CAN bus (1 Mbps); a separate ground-referenced line gives EMU and GSEMU instant umbilical disconnect-sense, independent of CAN.

3.1 RCU–EMU Radio Link

The link uses the HopeRF RFM95W module (Semtech SX1276 chipset) operating at 915.0 MHz with 23 dBm transmit power. Packets are simple binary structures (see Section ??). The RCU sends a **HEARTBEAT** packet at 5-second intervals when no other command is pending; the EMU echoes a **HEARTBEAT** response containing valve positions and battery voltage. Link loss is declared after 30 seconds without a response.

3.2 EMU–GSEMU–LCMU CAN Bus

EMU, GSEMU, and LCMU share one CAN bus (MCP2515 controllers, 1 Mbps, node IDs EMU = 1, GSEMU = 2, LCMU = 3) — addressed command/response only, no unsolicited broadcasts. This replaced an earlier wired GPIO CMD/STATUS handshake that carried Fill-valve coordination directly between EMU and GSEMU; that handshake’s two lines (AUX_IO_1/AUX_IO_2 on EMU’s MCP23X17 GPIO expander) are retired. Full command tables and protocol details for each unit are in their own reference documents (Section 6).

Separately, a ground-referenced disconnect-sense line on each side of the EMU–GSEMU umbilical (not CAN) gives both units instant physical-disconnect detection independent of bus traffic or timeout logic.

4 Shared Libraries

Common definitions reside in `KJO_Shared_Libraries` and are shared across all four firmware projects via PlatformIO library dependencies:

Library	Contents	Used by
<code>KJO_System_Config</code>	Platform IDs, version strings	RCU, EMU, GSEMU, LCMU
<code>KJO_Command_Defs</code>	LoRa command codes, packet structs	RCU, EMU
<code>KJO_CAN_Command_Defs</code>	CAN command codes, packet structs, node IDs	EMU, GSEMU, LCMU
<code>KJO_Radio</code>	Radio pin assignments, frequency	RCU, EMU
<code>KJO_RCU_Display</code>	TFT GUI functions and data types	RCU

5 Command Set

5.1 Packet Structures

Three binary packet types are used on the radio link:

- **Command_Payload** (4 bytes): `command` (2B), `param` (short, 2B). Used for most commands.
- **Heartbeat_Command_Packet** (6 bytes): `command` (2B), `RCU_voltage` (float, 4B). Sent by RCU as heartbeat.
- **Heartbeat_Response_Packet** (10 bytes): `command` (2B), `EMU_voltage` (float, 4B), `fill_pct`, `ox_pct`, `fuel_pct`, `dump_pct` (uint8, 1B each). Sent by EMU.
- **Command_Return_Packet**: `command` (2B), `status` (bool), `r_val` (short). Returned for non-heartbeat commands.

5.2 Command Table

Code	Name	Direction	param	Description
1	BEGIN_FILL	RCU→EMU	—	Command GSEMU to open Fill valve
2	STOP_FILL	RCU→EMU	—	Command GSEMU to close Fill valve
3	OPEN_OXIDIZER_FEED	RCU→EMU	—	Open Ox Feed valve fully
4	CLOSE_OXIDIZER_FEED	RCU→EMU	—	Close Ox Feed valve
5	OPEN_FUEL_FEED	RCU→EMU	—	Open Fuel Feed valve fully
6	CLOSE_FUEL_FEED	RCU→EMU	—	Close Fuel Feed valve
7	OXIDIZER_FLOW_PROP	RCU→EMU	%	Set Ox valve position (0–100%)
8	FUEL_FLOW_PROP	RCU→EMU	%	Set Fuel valve position (0–100%)
9	DUMP	RCU→EMU	—	Open Ox valve; vent pressurant through chamber
10	STOP_DUMP	RCU→EMU	—	Close Ox valve
11	START_ENGINE_FLOWS	RCU→EMU	—	Open both feed valves simultaneously
12	STOP_ENGINE_FLOWS	RCU→EMU	—	Close both feed valves simultaneously
13	ENABLE_LCO_CONTROL	RCU→EMU	—	Enable launch authority
14	START_ENGINE	RCU→EMU	—	Fire igniter and open valves
15	RESET	RCU→EMU	—	Reset EMU to safe state
16	READ_TANK_PRESSURE	RCU→EMU	—	Return oxidizer tank pressure
17	READ_CHAMBER_PRESSURE	RCU→EMU	—	Return chamber pressure
18	READ_TANK_TEMP	RCU→EMU	—	Return oxidizer tank temperature
19	READ_CHAMBER_TEMP	RCU→EMU	—	Return combustion chamber temperature
20	START_PRESSURE_RECORDING	RCU→EMU	—	Begin high-rate pressure logging
21	STOP_PRESSURE_RECORDING	RCU→EMU	—	End high-rate pressure logging
22	READ_VALVE_POSITION	RCU→EMU	valve#	Return servo position (0–100%)
23	HEARTBEAT	Bidirectional	—	Keepalive / link maintenance
24	RCU_STATUS	RCU→EMU	—	Request full status packet

6 Document References

Document	File	Contents
RCU Reference	RCU/KJO_RCU_Reference.pdf	Hardware, software, GUI
EMU Reference	EMU/KJO_EMU_Reference.pdf	Hardware, software, valve control
GSEMU Reference	GSEMU/KJO_GSEMU_Reference.pdf	Hardware, software, Fill valve
LCMU Reference	LCMU/KJO_LCMU_Reference.pdf	Hardware, software, CAN bus, load cells

7 Version History

Version	Date	Notes
0.1	March 2026	Initial release.